

SOUR VII

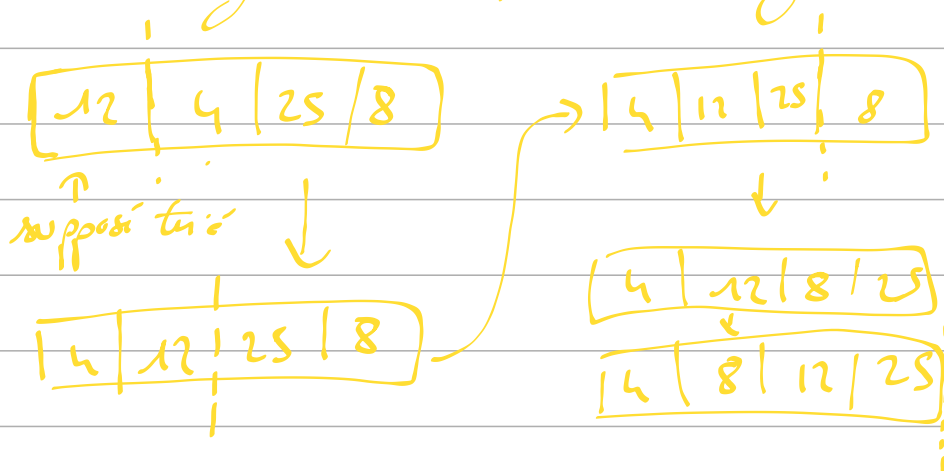
A) ALGORITHMIQUE

Notes en vrac de la lecture de l'énoncé:

- tri par insertion
 - laisser des places vides pour insérer proprement
 - ajout de bibliothèque et réviser/revoyé la lin
- ça ressemble à du hachage

Considérons un tableau T de taille n une puissance de deux, dont on suppose les éléments tous des entiers distincts.

1) Cons: algo du tri par insertion: (gauche, droite)



Au mieux: $\Theta(n)$: tableau déjà trié (n croissant)

En moyenne: $\Theta(n^2)$

Au pire: $\Theta(n^2)$: tableau trié en décroissant

→ cf. le cours.

$T \rightarrow B$
 $n \quad \alpha n$
 $\alpha > 1$
 $\uparrow$
 contient la
 m^{ème} éléments triés,
 $(\alpha-1)n$ cas vide

- i) insertion 2^h premiers éléments triés de T avec avant chacun 2 cas vide.
 ii) insérer les 2^{h-1} autres éléments à droite éventuellement les éléments présents vers la droite.

1) $T = [7, 2, 6, 8, 4, 5, 1, 3]$, $\alpha = 3$
 par 7 cas de T, il a fait 3 dans B.

$B_0 = [7]$

$h=1: B_1 = [\dots, \dots, 7]$
 $\downarrow$

$B_1 = [\dots, 2, 7]$

"tableau suffisamment grand"

$h=2: B_2 = [\dots, \dots, 2, \dots, \dots, 7, \dots]$
 $\downarrow$

$i=4$
 $B_2 = [\dots, \dots, 2, \dots, 6, 7, 8, \dots]$

$h=3: B_3 = [\dots, \dots, 7, \dots, \dots, 6, \dots, \dots, 7, \dots, \dots, 8, \dots]$
 $i=8$

$\downarrow$ insérer 4

ins(5) $(\dots, \dots, 2, \dots, 4, 6, \dots, \dots, 7, \dots, \dots, 8)$

ins(1) $(\dots, \dots, 2, \dots, 4, 5, 6, \dots, \dots, 7, \dots, \dots, 8)$

ins(2) $(\dots, \dots, 1, 2, \dots, 4, 5, 6, \dots, \dots, 7, \dots, \dots, 8)$

ins(3) $(\dots, \dots, 1, 2, 3, 4, 5, 6, \dots, \dots, 7, \dots, \dots, 8)$

3) * L'énacé fixe 8 espaces entre chaque entier dans (a).

① Donc par chaque entier de T, il faut 3 cases dans B.
($\alpha=3$)

* B_h doit être de taille au moins $3 \cdot 2^h$.

② Ici $P_2(B) = 3m$ par extension (m est une puissance de 2, $m = 2^{h_{max}}$)

4) eg. $h=2$, $B = [x, 1, 3, x, x, 5, x, 6, x, x, x, x]$

$\xrightarrow{\text{dispos}(B, 2)}$ $B_{3int} = [x, x, 1, x, x, 3, x, x, 5, x, x, 6, \dots]$

$x = \text{None}$.

def $\text{dispos}(B, h)$:

$n_elt = 2 \times (h-1)$

$pos = []$

for i in $\text{range}(P_2(B))$:

if $B[i]$ is not None: $pos.append(i)$

$pos = [1, 2, 5, 7]$

for j in $\text{range}(n_elt - 1, -1, -1)$:

$old_idx = pos[j]$

$new_idx = 3 * j + 2$

if ($new_idx \neq old_idx$):

$B[new_idx], B[old_idx] = B[old_idx], B[new_idx]$

$B[old_idx] = \text{None}$

5) Pour chaque étape on est à temps **linéaire**.

On a h étapes (à $h-1$).

① $h \in \mathbb{N}$: $n = 2^h$ i.e. $h = \lg n$

6) $choche(x, B, fin)$

idée: dichotomie:

! $\rightarrow$ on regarde l'élément le plus proche de B (milieu). (c'est en $O(1)$ ça)

! $\rightarrow$ si cet él est plus grand, on cherche à gauche, sinon à droite

! $\hookrightarrow$ jusqu'à trouver celui qui $>$ à gauche $<$ à droite.

! et on prend le case la plus à gauche (resp. la - à droite) qui est Nœud

7) Chaque étape est en $O(\lg(fin)) = O(\lg 2^h) = O(h \cdot \lg 2) = O(h)$
Et à chaque fois, on est en

$$\sum_{h=1}^{\lg n} 2^{h-1} O(h) \in O(n \lg n)$$

8) def $ins(x, B, i)$:
if $(B[i] \text{ is Nœud})$:
 $B[i] = x$
return

$vide = i$

while $vide < \text{len}(B)$ and $B[i]$ is not Nœud:

$vide += 1$

for j in range($vide, i, -1$):
 $B[j] = B[j-1]$

$B[i] = x$

10) tiBldio:

$$B = (\text{None}) * (3+n)$$

$$B(1) = T(1)$$

→ par h entre 0 et $\lg n$:

→ de'sperser (B, h) dans T

→ par elt entre 2^{h-1} , 2^h

idx = check (elt, B, 2^h)

insere (elt, B, idx)

return (x for x in B if x is not None)

11) ① Das 1^{er} par a_n , insere $\in \Theta(n)$

→ Cumulé: $\Theta(n)$

→ on est très proche de $\Theta(n)$ (et a_n est grand)

→ on aanti le $\Theta(n)$ on a plus les costs.

Donc on peut, tiBldio est $\in \Theta(\lg n)$

B) ANALYSE

On pose $\forall x \in \mathbb{R} \quad f(x) = \int_0^{+\infty} f(x,t) dt$ avec $f(x,t) = \frac{\sin(xt)}{t} e^{-t}$

1) f est continue sur $\mathbb{R} \times]0, +\infty[$ comme produit et composée de fonction qui le sont.

Continuité de f en $(x, 0)$:

Quand $t \rightarrow 0$, $f(x,t) \sim \frac{xt}{t} e^{-t} \xrightarrow{t \rightarrow 0} x$, donc f est prolongable.

Donc f est continue en $(x, 0)$

et continue. Donc f est continue sur $\mathbb{R} \times]0, +\infty[$.

Donc f est intégrable au voisinage de 0.

* Convergence def

$$|f(x,t)| = \left| \frac{\sin(xt)}{t} e^{-t} \right| \leq \frac{|xt|}{t} e^{-t} = |x| e^{-t} \sim_{t \rightarrow 0} e^{-t}, \text{ et } \int_0^{+\infty} e^{-t} dt \text{ cv.}$$

Conclusion: $\int_0^{+\infty} f(x,t) dt$ converge (absolument) donc pour tout $x \in \mathbb{R}$, on a que f est bien définie.

Remarque: dans la suite de l'exercice, on pose: $g(t) = e^{-t} \quad \forall t \in \mathbb{R}$.

2) i) $F(x)$ est simplement convergente (question 1)

ii) On a $\frac{\partial}{\partial x} f(x,t) = t \frac{\sin(xt)}{t} e^{-t} = \cos(xt) e^{-t}$ qui est continue sur $\mathbb{R} \times]0, +\infty[$

Tourner la page

iii.) On veut montrer que $\int_0^{+\infty} \cos(xt) e^{-t} dt$ converge normalement

$|\cos(xt) e^{-t}| \leq \frac{1}{e^t}$ et $\int_0^{+\infty} e^{-t} dt$ converge $\forall t \geq 0$ donc $F(x)$ converge normalement.

Par le dernier théorème du cours: $F \in \mathcal{C}'(\mathbb{R})$, et on a:

NB: On aurait pu, et surtout dans d'autres cas on doit, se restreindre à un intervalle du type $[-A, A]$, $A > 0$.

$$F'(x) = \int_0^{+\infty} \cos(xt) e^{-t} dt$$

Méthode 1: $\cos(xt) e^{-t} = \operatorname{Re}(e^{ixt}) e^{-t}$
 $= \operatorname{Re}(e^{(ix-1)t})$

Donc

$$F'(x) = \operatorname{Re} \left(\int_0^{+\infty} e^{(ix-1)t} dt \right)$$

$$\int_0^A e^{(ix-1)t} = \left[\frac{e^{(ix-1)t}}{ix-1} \right]_0^A = \frac{e^{ixA} e^{-A}}{ix-1} - \frac{1}{ix-1}$$

$\xrightarrow{A \rightarrow +\infty} 0$

donc $\operatorname{Re}(F'(x)) = \operatorname{Re}\left(\frac{1}{1-ix}\right)$

$$\text{Or } \frac{1}{1-ix} = \frac{1+ix}{(1-ix)(1+ix)} = \frac{1+ix}{1+x^2} = \frac{1}{1+x^2} + i \frac{x}{1+x^2}$$

Donc $F'(x) = \frac{1}{1+x^2}$

Méthode 2: faire une double IPP.

2) On a que $F'(x) = \frac{1}{1+x^2}$, on en déduit que $F(x) = \arctan(x) + C$.

Or $\lim_{x \rightarrow +\infty} F(x) = 0 \Rightarrow F(x) = \arctan(x)$.

C) LANGUAGE C

```
int main(void) {  
    int sum = 0; char str[12];  
    File * file = fopen("data.txt", "r");  
    if (file == NULL) return 1;  
  
    int cur = 0;  
    int tmp_val = 0;  
    while (cur = getc(file) != EOF) {  
        if (cur >= '0' && cur <= '9') {  
            tmp_val *= 10;  
            tmp_val += (cur - '0');  
        }  
        else {  
            sum += tmp_val;  
            tmp_val = 0;  
        }  
    }  
    sum += tmp_val; // le dernier nombre  
    fclose(file)  
  
    snprintf(str, sizeof(str), "%d", sum);  
    fprintf(stdout, "%s\n", str);  
  
    FILE * output = fopen("result.txt", "w");  
    if (output == NULL) return 1;  
    fprintf(output, "%s\n", str);  
}
```

suffisant pour un autre bono ou format string
(éventuellement: faire un memset(str, 0, sizeof(str))

// si c'est un chiffre